

Buone Pratiche di Codice

Buone Pratiche di Codice

Come scrivere codice C++ leggibile, manutenibile e professionale.

Scrivere codice che si capisce

Scrivere codice che funziona è il primo traguardo. Scrivere codice che **si capisce, si modifica facilmente e non crea problemi nel tempo** è il secondo — ed è quello che distingue un programmatore che cresce da uno che rimane bloccato.

Queste abitudini ti aiutano a:

- Trovare i bug più in fretta
- Capire il tuo stesso codice dopo settimane senza toccarlo
- Lavorare con altri programmatori senza causare confusione
- Evitare gli errori classici che fanno perdere tempo

Usa nomi che spiegano da soli

Il nome di una variabile o di una funzione deve dire **cosa rappresenta** o **cosa fa**:

```
// Scadente: cosa sono x, y, b?  
int x, y;  
double d;  
bool b;  
  
// Buono: si capisce subito  
int larghezza, altezza;  
double prezzoUnitario;  
bool maggiorenne;
```

Lo stesso vale per le funzioni:

```
// Scadente
void f(int a, int b) { return a + b; }

// Buono: si capisce cosa fa senza leggere il corpo
int calcolaSomma(int primo, int secondo) { return primo + secondo; }
```

Commenta il "perché", non il "cosa"

Il codice mostra già **cosa** fa. I commenti devono spiegare **perché** lo fa:

```
// Scadente: ridondante, il codice lo dice già
tentativi = tentativi + 1; // incrementa tentativi

// Buono: spiega il ragionamento
tentativi = tentativi + 1; // ogni risposta sbagliata aggiunge un tentativo
```

Documenta le funzioni complesse:

```
// Calcola il massimo comun divisore di due interi positivi.
// Precondizione: a > 0 e b > 0
// Usa l'algoritmo di Euclide (sottrazione ripetuta del resto).
int mcd(int a, int b) {
    while (b != 0) {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}
```

Ogni funzione fa una sola cosa

Se una funzione è troppo lunga o fa troppe cose, spezzala. Una funzione ben fatta si capisce in pochi secondi:

```

// Scadente: una funzione enorme che legge, calcola e stampa tutto insieme
void elaboraStudenti() {
    // 100 righe con tutto mescolato...
}

// Buono: ogni funzione ha una responsabilità precisa
void leggiVoti(vector<int>& voti);
double calcolaMedia(const vector<int>& voti);
int trovaMassimo(const vector<int>& voti);
void stampaReport(const string& nome, double media, int max);

void elaboraStudenti() {
    vector<int> voti;
    leggiVoti(voti);
    double media = calcolaMedia(voti);
    int max = trovaMassimo(voti);
    stampaReport("Alice", media, max);
}

```

Evita i "numeri magici"

Un numero scritto direttamente nel codice senza spiegazione si chiama "numero magico". Chi lo legge non capisce cosa rappresenta:

```

// Scadente: cosa sono 86400 e 3.14159?
if (secondi > 86400) {
    area = 3.14159 * r * r;
}

// Buono: i nomi spiegano tutto
const int SECONDI_PER_GIORNO = 86400;
const double PI = 3.14159265358979;

if (secondi > SECONDI_PER_GIORNO) {
    area = PI * r * r;
}

```

Inizializza sempre le variabili

In C++, una variabile non inizializzata contiene un valore casuale (qualunque cosa ci fosse prima in quella zona di memoria):

```
// Pericoloso: valore casuale!
int x;
cout << x;    // stampa qualcosa di imprevedibile

// Sicuro
int x = 0;
cout << x;    // stampa 0
```

Valida l'input dell'utente

Non fidarti mai di quello che l'utente inserisce:

```
int eta;
cout << "Inserisci eta: ";
cin >> eta;

// Scadente: accetta qualsiasi valore, anche -500 o 9999
cout << "Hai " << eta << " anni";

// Buono: controlla prima di usare
if (eta < 0 || eta > 150) {
    cout << "Eta non valida!" << endl;
    return 1;
}
cout << "Hai " << eta << " anni";
```

Evita le variabili globali

Le variabili globali rendono il codice difficile da capire: chiunque, in qualunque punto del programma, può modificarle. Meglio passare i dati come parametri:

```
// Scadente: la variabile globale può essere modificata ovunque
int contatore = 0;
void incrementa() { contatore++; }

// Meglio: i dati vengono passati esplicitamente
void incrementa(int& contatore) { contatore++; }
```

Usa **const** per i dati che non cambiano

const comunica le intenzioni e previene modifiche accidentali:

```
// Parametri che la funzione non deve modificare
void stampa(const string& nome) {
    cout << nome << endl;
    // nome = "altro"; // il compilatore ti ferma se provi
}

// Valori fissi
const int MAX_STUDENTI = 30;
const double PI = 3.14159;
```

Verifica sempre le operazioni che possono fallire

Non assumere che un'operazione sia andata a buon fine:

```
// Scadente: apre il file senza verificare
ifstream file("dati.txt");
string riga;
getline(file, riga); // e se il file non esiste?

// Buono: controlla prima di usare
ifstream file("dati.txt");
if (!file.is_open()) {
    cerr << "Errore: impossibile aprire 'dati.txt'" << endl;
    return 1;
}
string riga;
getline(file, riga); // ora è sicuro
```

Non ripetere il codice (DRY)

DRY significa "Don't Repeat Yourself" — non ripeterti. Se scrivi lo stesso codice in più punti, estrailo in una funzione:

```
// Scadente: stessa struttura ripetuta
cout << "=====" << endl;
cout << "Alice: 8.5" << endl;
cout << "=====" << endl;
cout << "=====" << endl;
cout << "Bob: 7.2" << endl;
cout << "=====" << endl;

// Buono: una funzione riutilizzabile
void stampaScheda(const string& nome, double voto) {
    cout << "=====" << endl;
    cout << nome << ": " << voto << endl;
    cout << "=====" << endl;
}

stampaScheda("Alice", 8.5);
stampaScheda("Bob", 7.2);
```

Testa con casi diversi

Quando scrivi una funzione, testala con:

- Valori normali (caso standard)
- Valori limite (il minimo e il massimo consentiti)
- Valori non validi (cosa succede se riceve dati sbagliati?)

```
// Testa la funzione massimo con vari casi
cout << massimo(5, 3) << endl;    // 5 – caso normale
cout << massimo(-1, -5) << endl;  // -1 – numeri negativi
cout << massimo(7, 7) << endl;    // 7 – valori uguali
```